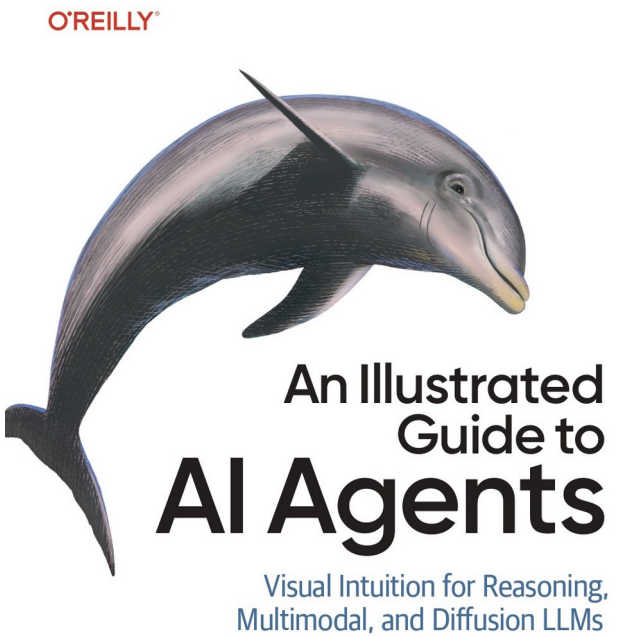
M1

What Is an AI Agent?

Module 1 — The Anatomy of an Agent



307307 — BI Methods for Generative AI

Agentic AI · University of Petra

Lecturer: _____ Date: _____

Learning Objectives

1. Define an AI agent and the four parts of the classic model: environment, sensors, actuators, agent program.
2. Explain why a plain LLM is not yet an agent — and what we add to make it one.
3. Describe the four augmentations in plain language: reasoning, memory, tools, planning & reflection.
4. Explain that autonomy is a spectrum, not an on/off switch.
5. Describe the TinyAgent you will build from scratch across the semester.

From Talking to Doing

Until recently, large language models (LLMs) mostly talked: you asked, they produced text. An AI agent takes the next step — given a goal, tools, and a bit of memory, it acts.

LLM

Plain LLM

Text in → text out

- Answers one question at a time
- No memory of past turns
- Cannot act on the world

AI

AI Agent

Goal in → actions out

- Works over many steps
- Remembers, uses tools, plans
- Searches, runs code, revises its plan

Throughout this course, "AI agent" means an agent whose brain is an LLM.

What Is an Agent? The Classic Definition

“An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators.”
— Russell & Norvig, Artificial Intelligence: A Modern Approach

- **Environment —** the world the agent works in
- **Sensors —** how it observes the environment
- **Actuators —** how it acts on the environment (its tools)
- **Agent program —** the “brain”: turns observations into actions

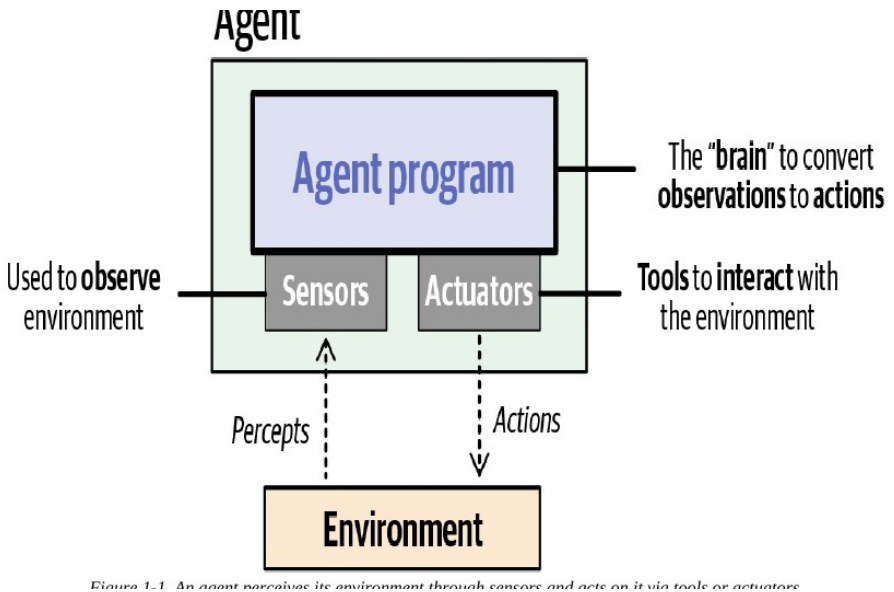


Figure 1-1, An Illustrated Guide to AI Agents (O'Reilly, 2026)

The Same Picture, in LLM Terms

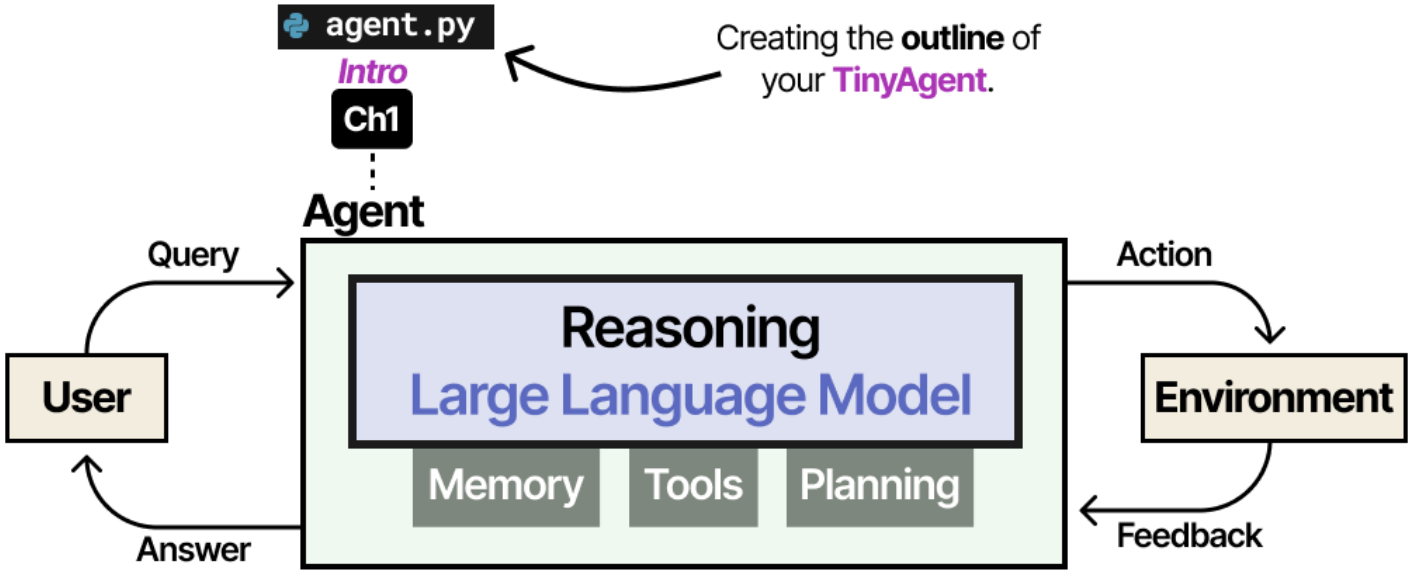


Figure from An Illustrated Guide to AI Agents (O'Reilly, 2026) — the backbone of this course

Brain

a reasoning LLM

Actuators

the agent's tools

Sensors

text, images, audio

User

starts the interaction

The Brain: From LLM to Reasoning LLM

LLM

- Predicts the next token (a sub-word piece of text).
- Feeds its own output back in and predicts again — this loop is autoregression.
- Writes a full answer one token at a time.
- **Fast and cheap.**

Reasoning LLM

- Trained to “think out loud” first — generates reasoning tokens before the answer.
- Structured thoughts make multi-step problems far easier.
- Better at planning, choosing tools, and recovering from mistakes.
- **Central to agents.**

Explored in the LLMs and Reasoning modules.

Augmenting the Brain — Memory

On its own, a reasoning LLM is still just text-in / text-out. First we give it memory.

Stateless by default	LLMs forget previous turns — each call starts fresh.
Simplest fix	Paste the conversation history back into the prompt.
Short- vs long-term	Real memory systems keep recent context and durable knowledge.
Context engineering	Too much context hurts — give the model the right information, not all of it.

Built in the Memory module (memory.py).

Augmenting the Brain — Tools (and MCP)

Tools let the agent act on its digital environment — but an LLM can only express the intent to act.

1. Ask

“What is 5.1 × 7.3?”

2. Intent

LLM outputs the text multiply(5.1, 7.3)

3. Execute

Your software parses it and runs the real function

4. Return

The result is fed back to the LLM

Two key ideas: the LLM's request to act is not authorization — your program decides whether to run it (a safety boundary). **The Model Context Protocol (MCP)** is a standard way to connect tools so different models can reuse them.

Built in the Tools module (tools.py + MCP).

Planning & Reflection

The final ingredient: deciding which steps to take, in what order, and when to stop.

1

Plan

break the task into smaller steps (task decomposition)

2

Act

carry out the next step

3

Observe

look at the result

4

Reflect

spot what went wrong and revise the plan

Repeat until the goal is reached. A reasoning LLM augmented with memory, tools, planning, and reflection is an AI agent.

Built in the Planning module (ReAct loop).

Autonomy Is a Spectrum

Prompting

one LLM call, one answer — not an agent

Fixed steps

a pipeline with a set order of steps — partly agentic

Autonomous agent

the model decides which steps to take, and when to stop

Goal-directed behavior + decisions = an agent. Guardrails keep autonomy to what the task needs.

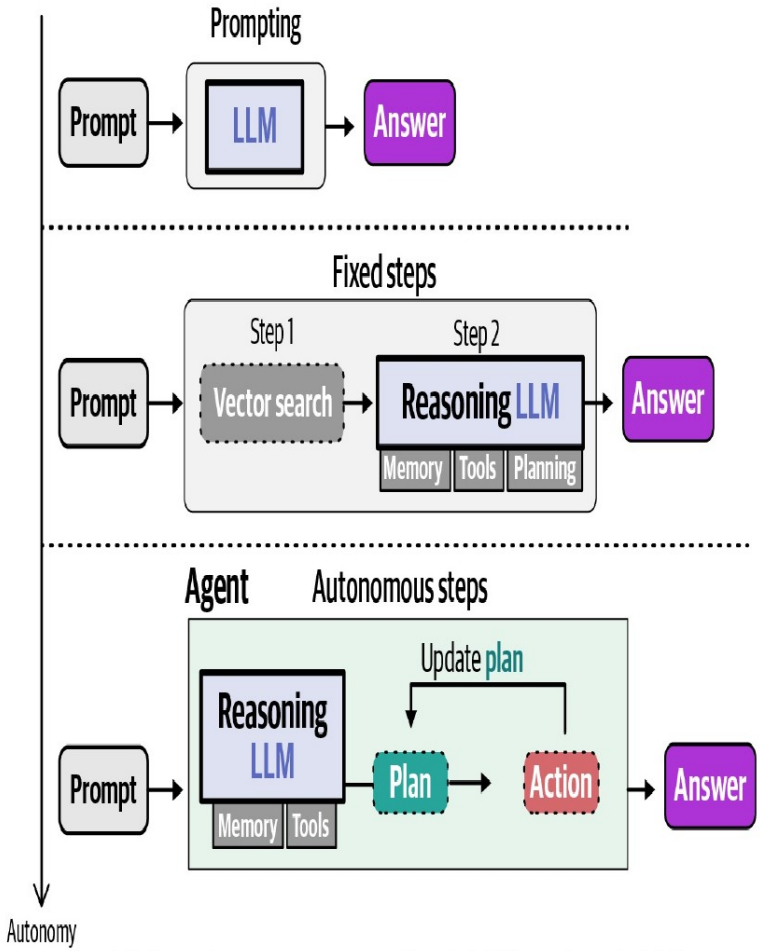


Figure 1-19. AI agents have more autonomy or agency than a single LLM generation or a fixed pipeline

Figure 1-19, An Illustrated Guide to AI Agents (O'Reilly, 2026)

What Agents Are Great For

Coding

Read a codebase, write new functions, run and test them. The goal is clear and results can often be checked automatically.

Deep research

Given a topic, search many sources and synthesise a report with little supervision.

Automation

Structure messy data across systems so people can actually work with it.

These thrive when the goal is clear but the exact path to reach it is not.

Use Responsibly

Because agents act — not just talk — they need care. This is where the Governance half of the course begins.

Human in the loop

Have a person authorize, check, or audit important decisions.

Guardrails

Limit autonomy to what the task actually needs — full control is often overkill, and risky.

Hallucinations

Agents are still LLMs and can state wrong things confidently. Add checks where correctness matters.

Evaluation matters too: judge an agent on its outcome and on its trajectory (the steps it took).

Beyond a Single Agent — The Road Ahead

Evaluation

Did the task get done (outcome) and were the steps sound (trajectory)?

Multi-agent

Several specialised agents collaborate, often under a supervisor.

Multimodal

Agents that can see and hear — images, audio, video, not only text.

Coding agents

Agents that run, test, and fix real code in a dedicated environment.

We meet these as a short survey near the end of the course — not as separate build modules.

What You Will Build — the TinyAgent

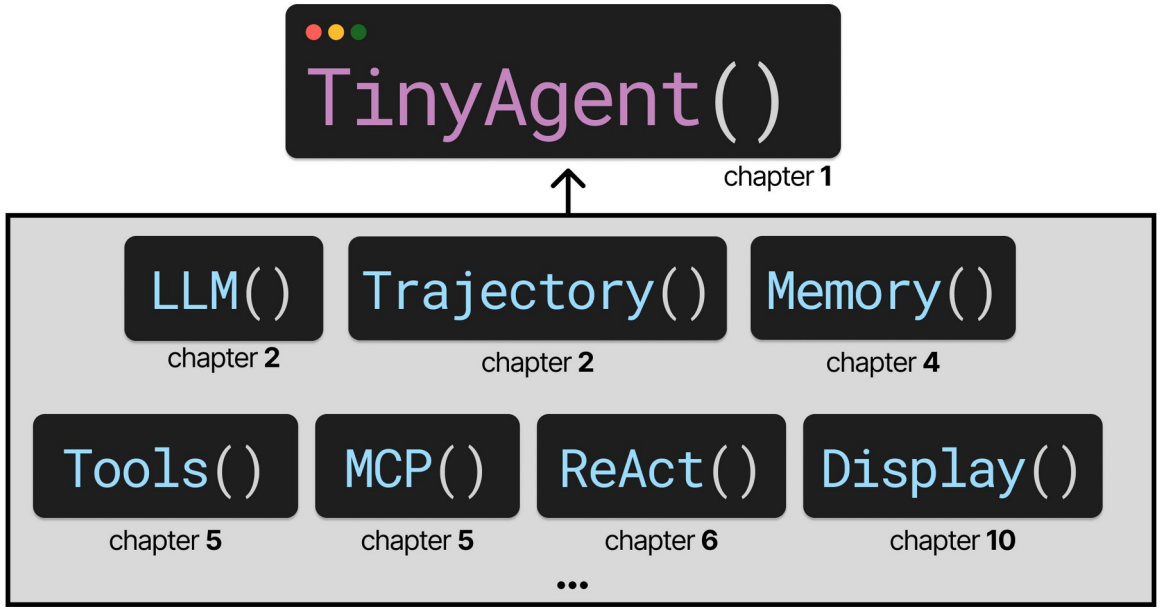


Figure from An Illustrated Guide to AI Agents (O'Reilly, 2026)

Built from scratch — no heavy frameworks. One module per topic, so you see how each part works.

Runs free on Google Gemini (the same setup as your Practical folder). Today's lab: the skeleton — and its first “brain”.

Course Roadmap — One Piece at a Time

- **M1** **What is an agent**
- **M2** LLMs & reasoning (the brain)
- **M3** Memory
- **M4** Tools & MCP
- **M5** Planning & reflection (ReAct)
- **+** Survey: multi-agent, multimodal, coding agents

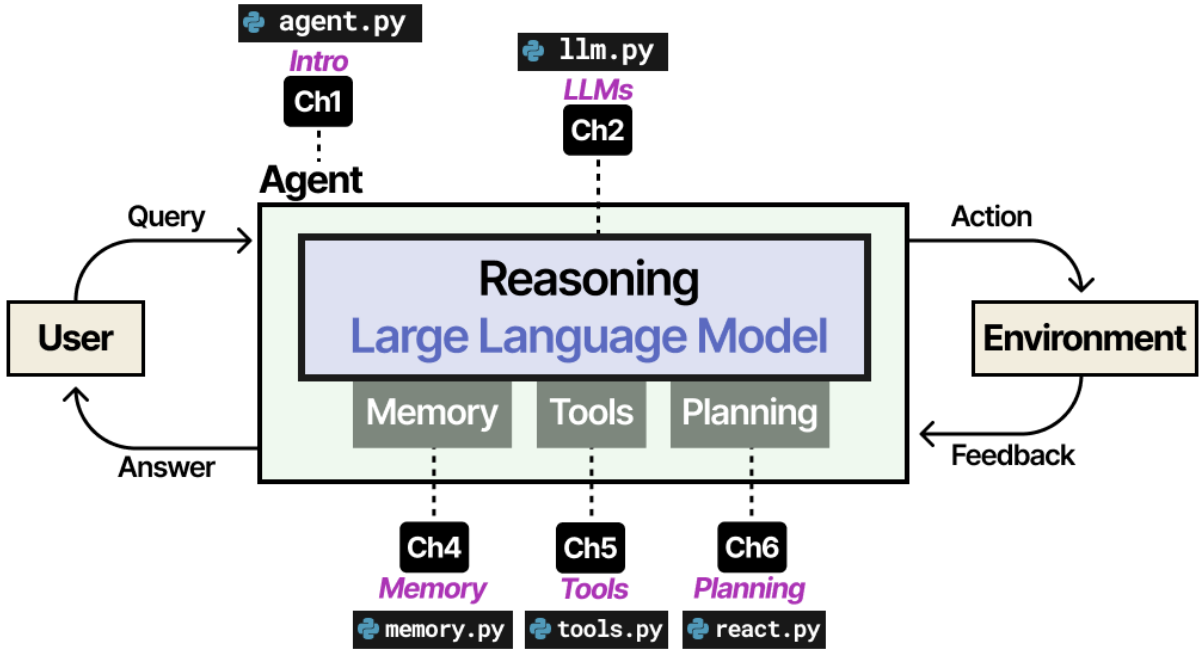


Figure from An Illustrated Guide to AI Agents (O'Reilly, 2026)

Key Takeaways

1. An agent perceives an environment through sensors and acts through actuators, guided by an agent program (the brain).
2. For us the brain is a reasoning LLM, augmented with memory, tools, and planning & reflection.
3. A plain LLM is not an agent — the augmentations and the action loop are what make it one.
4. Autonomy is a spectrum; guardrails and humans-in-the-loop matter.
5. You will build these ideas yourself as the TinyAgent, running on Gemini.

Discussion

Q1 Map the four parts of the classic agent definition onto an LLM-backed agent.

Q2 When an LLM outputs multiply(5.1, 7.3), has a tool been used? What still has to happen — and why is this a safety boundary?

Q3 Give an example of prompting, fixed steps, and an autonomous agent. What makes each more or less “agentic”?

Next: the LLMs module — giving the TinyAgent a proper brain.